MODULE 12 · DOMAIN 2 - ACCOUNT MANAGEMENT AND DATA GOVERNANCE

Domain 2 drill – six worked scenarios

No new content. Everything here comes from modules 8 to 11, in the form the exam uses.

DOMAIN 2 of 5 WEIGHT 20% MODULE 12 of 31

admin@sthithapragna.com

How to use this drill

- **Commit before I speak.** The pause is longer than in the teaching modules and it is there to be used.
- **Write down why.** The exam rephrases; a right answer for the wrong reason will not survive that.
- Domain 2 is **20%**, so about **20 of your 100 questions** come from this material.
- **Four or more of six** suggests the domain is solid. The final slide maps every question back to its module.

> ON THE EXAM
>
> Two misses in the same module is a signal. Two misses spread across four is normal.

A governance team creates a database role in the SALES database, grants it SELECT on several tables, and grants that database role directly to three analysts.

The statement granting the role to the analysts fails. The team then tries USE ROLE with the database role name in a session, which also fails.

What is the correct approach?

A Grant the database role to an account role, then grant that account role to the analysts.

B Database roles must be created by SECURITYADMIN. Recreate it using that role.

C Grant the database role to PUBLIC, which every user already holds.

D Enable secondary roles for the analysts, which allows database roles to be activated directly.

◆ PAUSE AND DECIDE

What is the correct approach?

A Grant it to an account role, then grant that to the analysts.

Database roles cannot be granted to users or activated in a session. Routing through an account role is the documented pattern.

B It must be created by SECURITYADMIN.

Who created it is irrelevant. The failure is about what a database role can be granted to, not who made it.

C Grant it to PUBLIC.

PUBLIC is an account role, so this would technically work and gives every user in the account the access. Not the intent.

D Enable secondary roles.

Secondary roles activate account roles the user already holds. They do not make a database role directly activatable.

WHERE THIS CAME FROM

Module 8. Both failures have the same cause: a database role cannot be granted to a user, and cannot be activated directly.

A security engineer creates a network rule of type IPV4 containing the corporate office CIDR range, in a schema called SECURITY.RULES. They confirm the rule exists with SHOW NETWORK RULES.

A week later, an audit finds that connections are still being accepted from arbitrary addresses worldwide.

What was missed?

A Network rules must be created in the SNOWFLAKE database to take effect account-wide.

B A network rule does nothing alone. It must be added to a network policy, which must then be activated.

C The rule type should have been AWSVPCEID, since IPV4 rules are advisory only.

D Network rules only apply to Snowsight sign-ins, not to driver connections.

◆ PAUSE AND DECIDE

What was missed?

A Rules must live in the SNOWFLAKE database.

Network rules are ordinary schema-level objects and can live in any schema. Their location is not the problem.

B It must be added to a policy, and the policy activated.

Rules hold identifiers. Policies hold rules in allowed and blocked lists, and do nothing until activated on an account, integration or user.

C IPV4 rules are advisory only.

Invented. IPV4 rules are enforced. Private connectivity rules take precedence over them, which is a different point.

D They only apply to Snowsight.

Network policies apply to all connections regardless of client or authentication method.

WHERE THIS CAME FROM

Module 9. Two objects, two steps. Creating the rule is the easy half; the policy and its activation are the half that enforces anything.

A privacy team enables automatic classification across a database with default settings, and asks Snowflake to auto-apply tags. Two weeks later an internal review finds a column of passport numbers that was never tagged, in a table where only about 0.4 percent of rows have a value in that column.

The review also finds several views exposing sensitive columns, none of which were classified.

Why were these missed?

A Classification only runs once. It must be re-run manually after any new data arrives.

B Auto-apply must be disabled for sparse columns, which are only ever recommended, never tagged.

C Classification samples up to 10,000 rows per table and excludes views from classification entirely.

D Passport numbers have no native semantic category, so no column of them can ever be classified.

◆ PAUSE AND DECIDE

Why were these missed?

A It only runs once.

Default behaviour reclassifies previously classified objects every 30 days. Re-running is not the issue.

B Sparse columns are only recommended.

Invented. auto_tag governs whether tags are applied at all; it does not vary by how sparse a column is.

C Samples 10,000 rows, and excludes views.

Both documented defaults, and each explains one half of the finding. A value in 0.4% of rows may not appear in the sample.

D No native category for passport numbers.

Custom categories can be created for anything not natively covered. It would also not explain the views.

WHERE THIS CAME FROM

Module 10. Two findings, two documented limits. Classification assists human judgement; it does not certify that nothing was missed.

A platform team builds an operational dashboard on SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY, intending to alert on-call staff within minutes when a query exceeds a runtime threshold.

In testing, alerts arrive reliably but always well over an hour after the offending query has finished. The dashboard query itself runs in seconds.

What is the cause, and what should they use?

A QUERY_HISTORY excludes queries still within their Time Travel window, which delays their appearance.

B The dashboard warehouse is too small. Increase its size so the view refreshes faster.

C ACCOUNT_USAGE only refreshes when queried by ACCOUNTADMIN. Change the dashboard's role.

D ACCOUNT_USAGE views have latency of up to 3 hours. Use the INFORMATION_SCHEMA table function for recent activity.

◆ PAUSE AND DECIDE

What is the cause, and what should they use?

A. Time Travel delays appearance.

Time Travel governs historical data in tables. It has nothing to do with metadata view latency.

B. The dashboard warehouse is too small.

The dashboard query already runs in seconds. Latency is in the metadata pipeline, not in the query.

C. It refreshes only for ACCOUNTADMIN.

Invented. The role governs whether you can read the view, not how current its data is.

D. ACCOUNT_USAGE latency. Use INFORMATION_SCHEMA.

Latency runs from 45 minutes to 3 hours, most views around 2. The Information Schema function has no latency at all.

WHERE THIS CAME FROM

Module 11. Same view name, opposite trade. ACCOUNT_USAGE buys a year of history at the price of hours of lag.

An analytics team owns several tables in a schema and wants to grant a partner team SELECT on one of them. The grant statement fails, although the analytics role holds OWNERSHIP on the table.

The schema was created using CREATE SCHEMA ... WITH MANAGED ACCESS. The analytics role does not hold MANAGE GRANTS and does not own the schema.

Why did the grant fail?

A OWNERSHIP does not include the ability to grant. A separate GRANT privilege must be held on the table.

B Managed access schemas prevent all grants. Access must be given through a secure view instead.

C In a managed access schema, object owners lose the ability to grant. Only the schema owner or a role with MANAGE GRANTS can.

D The partner team must first be granted USAGE on the schema by the analytics role, which it cannot do.

◆ PAUSE AND DECIDE

Why did the grant fail?

A OWNERSHIP does not include granting.

Normally it does. That is the discretionary half of the model, and a managed access schema is the exception to it.

B Managed access prevents all grants.

It centralises who may grant, not whether granting is possible. The schema owner grants freely.

C Owners lose the ability to grant. Schema owner or MANAGE GRANTS only.

Exactly what managed access does, and the scenario states the analytics role has neither qualification.

D USAGE on the schema must come first.

USAGE is genuinely required to reach the table, and it is not why this GRANT statement failed.

WHERE THIS CAME FROM

Modules 8 and 10 together. Ownership normally carries the right to grant. Managed access is the one place Snowflake switches that off.

A regulator asks a company to confirm, within the hour, which users read a specific customer column during the previous ninety minutes. The company runs Snowflake Standard Edition.

A data engineer proposes querying SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY and filtering to the last ninety minutes.

What is wrong with the proposal?

A ACCESS_HISTORY records writes only. QUERY_HISTORY should be used for reads instead.

B Nothing. ACCESS_HISTORY is available in all editions and ACCOUNT_USAGE has no latency.

C Only that the engineer needs ACCOUNTADMIN. Otherwise the approach is correct.

D Two problems: ACCESS_HISTORY requires Enterprise, and ACCOUNT_USAGE latency means the last 90 minutes may not be present.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A ACCESS_HISTORY records writes only.

It records reads and writes at column level. Reads are precisely what it is for.

B Nothing is wrong.

Wrong on both counts. ACCESS_HISTORY is Enterprise, and every ACCOUNT_USAGE view has latency.

C Just needs ACCOUNTADMIN.

True that ACCOUNT_USAGE is ACCOUNTADMIN by default, and it is the least of the problems here.

D Enterprise required, and latency hides the last 90 minutes.

Two independent blockers. ACCESS_HISTORY is Enterprise, and ACCOUNT_USAGE lags 45 minutes to 3 hours.

WHERE THIS CAME FROM

Modules 2, 10 and 11. Edition, then latency. Fixing either one alone still leaves the request unanswerable.

Where each question came from

QUESTION	MODULE TO REVISIT IF YOU MISSED IT
1. Database role cannot be granted to a user	Module 8 - account roles and database roles
2. Network rule with no policy	Module 9 - rules are schema objects, policies are account objects
3. Classification missed a sparse column and all views	Module 10 - classification profiles and the defaults
4. Dashboard alerts arrive an hour late	Module 11 - ACCOUNT_USAGE latency
5. Owner cannot grant in a managed access schema	Modules 8 and 10 together
6. ACCESS_HISTORY within the hour on Standard	Modules 2, 10 and 11 together

ON THE EXAM

Questions 5 and 6 span more than one module. Those are the ones worth re-reading even if you got them right.

What this drill was really testing

- **Two objects, two steps.** A rule needs a policy. A database role needs an account role. A policy needs activating.
- **Documented defaults you did not choose.** 10,000-row sampling, views excluded, hours of latency, ACCOUNTADMIN-only.
- **Exceptions to a rule you already learned.** Owners can grant, except in a managed access schema.
- **Constraints that stack.** Question 6 had two blockers. Fixing one leaves you no better off.

> ON THE EXAM
>
> Next: Domain 3. Data loading, unloading and connectivity at 18%, beginning with stages and file formats.